RETAILMART V3 · SQL

Pivoting, Unpivoting & FILTER Clause

— Siraj Sir

📱 WhatsApp Announcement

Pivoting, Unpivoting & FILTER Clause

Welcome to earlier: Analytics Power Tools. Today you learn to reshape data -- turning rows into columns (pivot) and columns into rows (unpivot). This is how you build the cross-tab reports that stakeholders love.

Part 1: Pivoting with CASE WHEN (Conditional Aggregation)
Part 2: The FILTER Clause -- Cleaner Pivots
Part 3: Unpivoting -- Columns Back to Rows

Every 'revenue by month with one column per category' report is a pivot. Every 'compare metrics side by side' table is a pivot. Today you learn both the classic CASE WHEN approach and PostgreSQL's cleaner FILTER(WHERE) syntax.

TODAY'S AGENDA (2 Hours)

Section	Time	Topics
Part 1: Pivoting with CASE WHEN	2 Hours	Conditional aggregation pattern, Revenue by month with category columns, Count by status pivot
Part 2: FILTER Clause		FILTER(WHERE) syntax, Cleaner alternative to CASE WHEN, When to use FILTER vs CASE WHEN
Part 3: Unpivoting		UNION ALL for unpivoting, When to unpivot in SQL vs BI tool

YOUR SQL JOURNEY

- ✓ SQL Intro
- ✓ Setup & RetailMart
- ✓ DDL & Data Types
- ✓ Constraints & DML
- ✓ Filtering & Sorting
- ✓ Scalar Functions
- ✓ Conditional Logic
- ✓ Aggregation
- ✓ Joins Part 1
- ✓ Subqueries Part 1
- ✓ Subqueries & CTEs
- ✓ Database Concepts
- ✓ Review & Practice
- ✓ Window Funcs Part 1
- ✓ Aggregation & Frames
- ✓ LAG & LEAD
- ✓ Query Plans
- ✓ Indexing

RECAP

What We Covered Yesterday

- B-Tree indexes: CREATE INDEX for faster lookups
- Composite indexes and leftmost prefix rule
- Partial indexes for active-records patterns
- When indexes help vs when Seq Scan is faster

STORY

The Revenue Matrix

Your manager asks: 'Give me monthly revenue broken down by payment mode -- one column per mode.' Your data has one row per order with a payment_mode column. But the manager wants a table where each ROW is a month and each COLUMN is a payment mode. That transformation -- rows into columns -- is called pivoting.

DEFINITION

Pivot = Rows to Columns

- Takes distinct values from one column and turns them into separate columns
- Each new column contains an aggregated value (SUM, COUNT, etc.)
- PostgreSQL has no PIVOT keyword -- use CASE WHEN inside aggregates

Pivot Template

```
SELECT
  group_column,
  SUM(CASE WHEN pivot_col = 'val1' THEN measure END) AS val1,
  SUM(CASE WHEN pivot_col = 'val2' THEN measure END) AS val2,
  SUM(CASE WHEN pivot_col = 'val3' THEN measure END) AS val3
FROM table
GROUP BY group_column;
```

CASE WHEN returns the measure for matching rows, NULL for non-matching. SUM ignores NULLs, so each column only totals its own category.

Revenue Pivot

```
SELECT
  DATE_TRUNC('month', order_date) AS order_month,
  SUM(CASE WHEN order_status = 'Delivered'
    THEN net_total END)        AS delivered_rev,
  SUM(CASE WHEN order_status = 'Shipped'
    THEN net_total END)        AS shipped_rev,
  SUM(CASE WHEN order_status = 'Cancelled'
    THEN net_total END)        AS cancelled_rev,
  SUM(net_total)               AS total_rev
FROM sales.orders
GROUP BY DATE_TRUNC('month', order_date)
ORDER BY order_month;
```

Count Pivot

```
SELECT
  dr.region_name,
  COUNT(CASE WHEN o.order_status = 'Delivered' THEN 1 END)
    AS delivered,
  COUNT(CASE WHEN o.order_status = 'Shipped' THEN 1 END)
    AS shipped,
  COUNT(CASE WHEN o.order_status = 'Cancelled' THEN 1 END)
    AS cancelled,
  COUNT(CASE WHEN o.order_status = 'Returned' THEN 1 END)
    AS returned,
  COUNT(*) AS total
FROM sales.orders o
JOIN stores.stores s ON o.store_id = s.store_id
JOIN core.dim_region dr ON s.region_id = dr.region_id
GROUP BY dr.region_name
ORDER BY dr.region_name;
```

PRO TIP

PRO TIP

For COUNT pivots, use COUNT(CASE WHEN … THEN 1 END) -- the THEN value does not matter as long as it is not NULL.
For SUM pivots, THEN must be the actual measure column.

Practice 1: Revenue by Payment Mode

EASY

SCENARIO: The finance team wants monthly revenue with one column per payment mode.

TASK: Pivot monthly revenue by payment_mode_id. Show columns for at least 3 payment modes plus a total column. Use the payment_modes table to identify the mode names.

HINT: JOIN orders to finance.payment_modes to get mode names. Then SUM(CASE WHEN mode_name = 'X' THEN net_total END) for each mode.

Answer

✅ ANSWER

```
SELECT
  DATE_TRUNC('month', o.order_date) AS month,
  SUM(CASE WHEN pm.mode_name = 'Credit Card'
    THEN o.net_total END)        AS credit_card,
  SUM(CASE WHEN pm.mode_name = 'Debit Card'
    THEN o.net_total END)        AS debit_card,
  SUM(CASE WHEN pm.mode_name = 'UPI'
    THEN o.net_total END)        AS upi,
  SUM(CASE WHEN pm.mode_name = 'Cash'
    THEN o.net_total END)        AS cash,
  SUM(o.net_total)               AS total
FROM sales.orders o
JOIN finance.payment_modes pm
  ON o.payment_mode_id = pm.mode_id
GROUP BY DATE_TRUNC('month', o.order_date)
ORDER BY month;
```

Practice 2: Employee Count by Department Tenure

MEDIUM

SCENARIO: HR wants a table showing employee counts by department, with columns for tenure buckets: <1yr, 1-3yr, 3-5yr, 5yr+.

TASK: Pivot employee counts by department (rows) and tenure bucket (columns). Calculate tenure from joining_date to CURRENT_DATE.

HINT: Use EXTRACT(YEAR FROM AGE(CURRENT_DATE, joining_date)) to get years of tenure. Then CASE WHEN to bucket and COUNT.

Answer

✅ ANSWER

```
SELECT
  d.dept_name,
  COUNT(CASE WHEN EXTRACT(YEAR FROM
    AGE(CURRENT_DATE, joining_date)) < 1
    THEN 1 END)                    AS under_1yr,
  COUNT(CASE WHEN EXTRACT(YEAR FROM
    AGE(CURRENT_DATE, joining_date)) BETWEEN 1 AND 2
    THEN 1 END)                    AS yr_1_to_3,
  COUNT(CASE WHEN EXTRACT(YEAR FROM
    AGE(CURRENT_DATE, joining_date)) BETWEEN 3 AND 4
    THEN 1 END)                    AS yr_3_to_5,
  COUNT(CASE WHEN EXTRACT(YEAR FROM
    AGE(CURRENT_DATE, joining_date)) >= 5
    THEN 1 END)                    AS yr_5_plus,
  COUNT(*)                         AS total
FROM stores.employees e
JOIN core.dim_department d ON e.dept_id = d.dept_id
GROUP BY d.dept_name
ORDER BY d.dept_name;
```

STORY

The Cleaner Way

The CASE WHEN pivot pattern works but is verbose. PostgreSQL offers a cleaner syntax: FILTER(WHERE). It does exactly the same thing -- filters rows before aggregation -- but reads like plain English. It is a PostgreSQL extension (not standard SQL), but it is widely used.

FILTER Clause

```
-- Basic FILTER

 SUM(amount) FILTER (WHERE status = 'X')
   -- Same as SUM(CASE WHEN status = 'X'
   --   THEN amount END)

-- COUNT with FILTER

 COUNT(*) FILTER (WHERE status = 'X')
   -- Counts only rows matching the filter

-- Multiple FILTERs

 SUM(amount) FILTER (WHERE cat = 'A') AS a,
 SUM(amount) FILTER (WHERE cat = 'B') AS b
```

Comparison

```
-- CASE WHEN approach (verbose):
SELECT
  region,
  SUM(CASE WHEN order_status = 'Delivered'
    THEN net_total END) AS delivered,
  SUM(CASE WHEN order_status = 'Cancelled'
    THEN net_total END) AS cancelled
FROM ...

-- FILTER approach (cleaner):
SELECT
  region,
  SUM(net_total)
    FILTER (WHERE order_status = 'Delivered') AS delivered,
  SUM(net_total)
    FILTER (WHERE order_status = 'Cancelled') AS cancelled
FROM ...

-- Identical results. FILTER is just easier to read.
```

Complex FILTER

```
SELECT
  DATE_TRUNC('month', order_date) AS month,
  COUNT(*) AS total_orders,
  COUNT(*) FILTER (WHERE net_total > 10000)
    AS high_value_orders,
  SUM(net_total)
    FILTER (WHERE order_status = 'Delivered'
      AND net_total > 5000)    AS premium_delivered_rev,
  ROUND(AVG(net_total)
    FILTER (WHERE order_status != 'Cancelled')::NUMERIC, 2)
    AS avg_non_cancelled
FROM sales.orders
GROUP BY DATE_TRUNC('month', order_date)
ORDER BY month;
```

PRO TIP

PRO TIP

FILTER works with ANY aggregate: SUM, COUNT, AVG, MIN, MAX, STRING_AGG, even PERCENTILE_CONT. It is not limited to pivoting -- use it whenever you need a conditional aggregate.

Practice 3: Revenue Pivot with FILTER

MEDIUM

SCENARIO: Rewrite the payment mode pivot from Practice 1 using FILTER instead of CASE WHEN.

TASK: Show monthly revenue with columns for Credit Card, UPI, and Cash using FILTER(WHERE). Compare the readability to the CASE WHEN version.

HINT: SUM(o.net_total) FILTER (WHERE pm.mode_name = 'Credit Card') AS credit_card

Answer

✅ ANSWER

```
SELECT
  DATE_TRUNC('month', o.order_date) AS month,
  SUM(o.net_total)
    FILTER (WHERE pm.mode_name = 'Credit Card')
    AS credit_card,
  SUM(o.net_total)
    FILTER (WHERE pm.mode_name = 'UPI')
    AS upi,
  SUM(o.net_total)
    FILTER (WHERE pm.mode_name = 'Cash')
    AS cash,
  SUM(o.net_total) AS total
FROM sales.orders o
JOIN finance.payment_modes pm
  ON o.payment_mode_id = pm.mode_id
GROUP BY DATE_TRUNC('month', o.order_date)
ORDER BY month;
```

Practice 4: Pivot with Percent-of-Row

HARD

SCENARIO: The CFO wants to see not just revenue per status, but each status as a percentage of the row total.

TASK: For each region, show delivered revenue, cancelled revenue, and each as a percentage of the region's total revenue. Round percentages to 1 decimal.

HINT: Use FILTER for the numerator, plain SUM for the denominator. Divide and multiply by 100.

Answer (1/2)

✅ ANSWER

```
SELECT
  dr.region_name,
  SUM(o.net_total) AS total_rev,
  SUM(o.net_total)
    FILTER (WHERE o.order_status = 'Delivered')
    AS delivered_rev,
  ROUND(
    SUM(o.net_total)
      FILTER (WHERE o.order_status = 'Delivered')
    / NULLIF(SUM(o.net_total), 0) * 100, 1
  ) AS delivered_pct,
  SUM(o.net_total)
    FILTER (WHERE o.order_status = 'Cancelled')
    AS cancelled_rev,
  ROUND(
    SUM(o.net_total)
      FILTER (WHERE o.order_status = 'Cancelled')
    / NULLIF(SUM(o.net_total), 0) * 100, 1
  ) AS cancelled_pct
FROM sales.orders o
JOIN stores.stores s ON o.store_id = s.store_id
JOIN core.dim_region dr ON s.region_id = dr.region_id
```

Answer (2/2)

✅ ANSWER

```
GROUP BY dr.region_name
ORDER BY total_rev DESC;
```

STORY

The BI Tool Prefers Long Format

Pivoted data is great for spreadsheets. But BI tools like Tableau, Power BI, and Metabase prefer 'long format' -- one metric per row. If your source data is already pivoted (columns = Q1, Q2, Q3, Q4), you need to unpivot it back to rows before the BI tool can chart it properly.

Unpivot Example

```sql
-- Source: wide quarterly data
-- store_id | q1_rev | q2_rev | q3_rev | q4_rev

-- Unpivot to: store_id | quarter | revenue
SELECT store_id, 'Q1' AS quarter, q1_rev AS revenue
FROM quarterly_data
UNION ALL
SELECT store_id, 'Q2', q2_rev
FROM quarterly_data
UNION ALL
SELECT store_id, 'Q3', q3_rev
FROM quarterly_data
UNION ALL
SELECT store_id, 'Q4', q4_rev
FROM quarterly_data
ORDER BY store_id, quarter;
```

Each UNION ALL arm takes one column and turns it into a row with a label. The result is long format with one row per store per quarter.

When to Pivot in SQL vs BI Tool

- Pivot in SQL: when the result goes into a spreadsheet or static report
- Leave long in SQL, pivot in BI tool: when feeding Tableau/Power BI/Metabase
- Unpivot in SQL: when source data arrives in wide format but you need long for analysis

COMMON MISTAKE

Common Pivoting Mistakes

Forgetting to handle NULLs: CASE WHEN returns NULL for non-matching rows. SUM and COUNT ignore NULLs, but if you use COALESCE to show 0 instead, be intentional about it.

Hardcoding values that change: If payment modes change, your CASE WHEN columns break. Consider whether dynamic pivoting (in the BI layer) is better.

Unpivoting with UNION (not UNION ALL): UNION removes duplicates -- which is almost never what you want in an unpivot. Always use UNION ALL.

Practice 5: Unpivot a KPI Table

MEDIUM

SCENARIO: The analytics team stores KPIs in a wide table with columns for each metric. The BI tool needs it in long format.

TASK: Given the pivoted query result below (region, total_orders, total_revenue, avg_order_value), unpivot it into: region, metric_name, metric_value.

HINT: Three UNION ALL arms -- one per metric column.

Source Query

✅ ANSWER

```
-- Wide format (from a CTE or subquery):
-- region | total_orders | total_revenue | avg_order_value
-- North  | 12000        | 45000000      | 3750
-- South  | 15000        | 52000000      | 3467
```

Answer

✅ ANSWER

```
WITH kpis AS (
  SELECT
    dr.region_name AS region,
    COUNT(*)                        AS total_orders,
    SUM(o.net_total)              AS total_revenue,
    ROUND(AVG(o.net_total)::NUMERIC, 2)
                                    AS avg_order_value
  FROM sales.orders o
  JOIN stores.stores s ON o.store_id = s.store_id
  JOIN core.dim_region dr ON s.region_id = dr.region_id
  GROUP BY dr.region_name
)
SELECT region, 'total_orders' AS metric,
       total_orders::NUMERIC AS value FROM kpis
UNION ALL
SELECT region, 'total_revenue',
       total_revenue FROM kpis
UNION ALL
SELECT region, 'avg_order_value',
       avg_order_value FROM kpis
ORDER BY region, metric;
```

Practice 6: Category x Month Revenue Matrix

CRAZY

SCENARIO: Build the classic 'category vs month' revenue matrix that every retail analyst needs.

TASK: Build a pivoted table where rows = product categories and columns = months (Jan through Dec 2025). Each cell shows the total revenue. Use FILTER for the pivot. Include a total column.

HINT: FILTER(WHERE DATE_TRUNC('month', o.order_date) = '2025-01-01') for January. Use TO_CHAR(DATE_TRUNC(...), 'Mon') if you want month labels.

Answer (1/2)

✅ ANSWER

```
SELECT
  cat.category_name,
  SUM(oi.quantity * oi.unit_price)
    FILTER (WHERE EXTRACT(MONTH FROM o.order_date) = 1)
    AS jan,
  SUM(oi.quantity * oi.unit_price)
    FILTER (WHERE EXTRACT(MONTH FROM o.order_date) = 2)
    AS feb,
  SUM(oi.quantity * oi.unit_price)
    FILTER (WHERE EXTRACT(MONTH FROM o.order_date) = 3)
    AS mar,
  SUM(oi.quantity * oi.unit_price)
    FILTER (WHERE EXTRACT(MONTH FROM o.order_date) = 4)
    AS apr,
  SUM(oi.quantity * oi.unit_price)
    FILTER (WHERE EXTRACT(MONTH FROM o.order_date) = 5)
    AS may,
  SUM(oi.quantity * oi.unit_price)
    FILTER (WHERE EXTRACT(MONTH FROM o.order_date) = 6)
    AS jun,
  SUM(oi.quantity * oi.unit_price) AS total
FROM sales.order_items oi
```

Answer (2/2)

✅ ANSWER

```
JOIN sales.orders o ON oi.order_id = o.order_id
JOIN products.products p ON oi.prod_id = p.product_id
JOIN core.dim_brand b ON p.brand_id = b.brand_id
JOIN core.dim_category cat
  ON b.category_id = cat.category_id
WHERE o.order_status = 'Delivered'
  AND o.order_date >= '2025-01-01'
  AND o.order_date < '2025-07-01'
GROUP BY cat.category_name
ORDER BY total DESC;
```

INTERVIEW

Q: How do you pivot data in PostgreSQL without a PIVOT keyword?

A: Use conditional aggregation: SUM(CASE WHEN col = 'value' THEN measure END) for each pivot column. Or use the FILTER clause: SUM(measure) FILTER (WHERE col = 'value'). Both produce the same result. FILTER is PostgreSQL-specific but more readable.

INTERVIEW

Q: What is the difference between FILTER and CASE WHEN in aggregates?

A: Functionally identical. FILTER(WHERE condition) is syntactic sugar for CASE WHEN condition THEN value END inside an aggregate. FILTER is cleaner to read and PostgreSQL-specific. CASE WHEN works in all SQL dialects. Use FILTER in PostgreSQL, CASE WHEN for portable SQL.

HW 1: Order Count by Region x Status

EASY

SCENARIO: Regional managers want a quick overview of order distribution.

TASK: Pivot order counts by region (rows) and status (columns) using FILTER. Include Delivered, Shipped, Cancelled, and Out for Delivery.

Answer

✅ ANSWER

```
SELECT
  dr.region_name,
  COUNT(*) FILTER (WHERE o.order_status = 'Delivered')
    AS delivered,
  COUNT(*) FILTER (WHERE o.order_status = 'Shipped')
    AS shipped,
  COUNT(*) FILTER (WHERE o.order_status = 'Cancelled')
    AS cancelled,
  COUNT(*) FILTER (WHERE o.order_status = 'Out for Delivery')
    AS out_for_delivery,
  COUNT(*) AS total
FROM sales.orders o
JOIN stores.stores s ON o.store_id = s.store_id
JOIN core.dim_region dr ON s.region_id = dr.region_id
GROUP BY dr.region_name
ORDER BY dr.region_name;
```

HW 2: Pivot Order Count by Week x Status

MEDIUM

SCENARIO: Operations wants weekly order counts with a column per status bucket.

TASK: Show ISO week number, and columns for each status using FILTER. Only show weeks in 2025.

Answer

✅ ANSWER

```
SELECT
  EXTRACT(WEEK FROM order_date)::INT AS week_num,
  COUNT(*) FILTER (WHERE order_status = 'Delivered')
    AS delivered,
  COUNT(*) FILTER (WHERE order_status = 'Shipped')
    AS shipped,
  COUNT(*) FILTER (WHERE order_status = 'Cancelled')
    AS cancelled,
  COUNT(*) AS total
FROM sales.orders
WHERE order_date >= '2025-01-01'
  AND order_date < '2026-01-01'
GROUP BY EXTRACT(WEEK FROM order_date)
ORDER BY week_num;
```

HW 3: Unpivot Employee Metrics

MEDIUM

SCENARIO: HR has per-department metrics in wide format. The BI tool needs long format.

TASK: From a CTE that calculates department, headcount, avg_salary, and avg_tenure, unpivot into: department, metric_name, metric_value.

Answer (1/2)

✅ ANSWER

```
WITH dept_metrics AS (
  SELECT
    d.dept_name AS department,
    COUNT(*)                          AS headcount,
    ROUND(AVG(e.salary)::NUMERIC, 0)
                                      AS avg_salary,
    ROUND(AVG(EXTRACT(YEAR FROM
      AGE(CURRENT_DATE, e.joining_date)))::NUMERIC, 1)
                                      AS avg_tenure
  FROM stores.employees e
  JOIN core.dim_department d ON e.dept_id = d.dept_id
  GROUP BY d.dept_name
)
SELECT department, 'headcount' AS metric,
       headcount::NUMERIC AS value
FROM dept_metrics
UNION ALL
SELECT department, 'avg_salary', avg_salary
FROM dept_metrics
UNION ALL
SELECT department, 'avg_tenure', avg_tenure
FROM dept_metrics
```

Answer (2/2)

✅ ANSWER

```
ORDER BY department, metric;
```

HW 4: Revenue by Day-of-Week

HARD

SCENARIO: Operations wants to know which day of the week generates the most revenue.

TASK: Pivot total delivered revenue by store region (rows) with one column per day of the week (Mon, Tue, Wed, Thu, Fri, Sat, Sun). Use FILTER with EXTRACT(DOW FROM order_date).

Answer (1/2)

✅ ANSWER

```
SELECT
  dr.region_name,
  SUM(o.net_total)
    FILTER (WHERE EXTRACT(DOW FROM o.order_date) = 1)
    AS mon,
  SUM(o.net_total)
    FILTER (WHERE EXTRACT(DOW FROM o.order_date) = 2)
    AS tue,
  SUM(o.net_total)
    FILTER (WHERE EXTRACT(DOW FROM o.order_date) = 3)
    AS wed,
  SUM(o.net_total)
    FILTER (WHERE EXTRACT(DOW FROM o.order_date) = 4)
    AS thu,
  SUM(o.net_total)
    FILTER (WHERE EXTRACT(DOW FROM o.order_date) = 5)
    AS fri,
  SUM(o.net_total)
    FILTER (WHERE EXTRACT(DOW FROM o.order_date) = 6)
    AS sat,
  SUM(o.net_total)
    FILTER (WHERE EXTRACT(DOW FROM o.order_date) = 0)
```

Answer (2/2)

✅ ANSWER

```
    AS sun
FROM sales.orders o
JOIN stores.stores s ON o.store_id = s.store_id
JOIN core.dim_region dr ON s.region_id = dr.region_id
WHERE o.order_status = 'Delivered'
GROUP BY dr.region_name
ORDER BY dr.region_name;
```

HW 5: Pivot Returning Percent-of-Total

HARD

SCENARIO: Show each product category's share of total revenue per quarter.

TASK: For each category, show Q1 and Q2 revenue as a percentage of that quarter's total (across all categories). Use FILTER and window functions.

Answer (1/2)

✅ ANSWER

```
WITH quarterly AS (
  SELECT
    cat.category_name,
    SUM(oi.quantity * oi.unit_price)
      FILTER (WHERE EXTRACT(QUARTER
        FROM o.order_date) = 1)     AS q1_rev,
    SUM(oi.quantity * oi.unit_price)
      FILTER (WHERE EXTRACT(QUARTER
        FROM o.order_date) = 2)     AS q2_rev
  FROM sales.order_items oi
  JOIN sales.orders o ON oi.order_id = o.order_id
  JOIN products.products p ON oi.prod_id = p.product_id
  JOIN core.dim_brand b ON p.brand_id = b.brand_id
  JOIN core.dim_category cat
    ON b.category_id = cat.category_id
  WHERE o.order_status = 'Delivered'
    AND o.order_date >= '2025-01-01'
    AND o.order_date < '2025-07-01'
  GROUP BY cat.category_name
)
SELECT
  category_name,
```

Answer (2/2)

✅ ANSWER

```
  q1_rev,
  ROUND(q1_rev / NULLIF(SUM(q1_rev) OVER (), 0)
    * 100, 1) AS q1_pct,
  q2_rev,
  ROUND(q2_rev / NULLIF(SUM(q2_rev) OVER (), 0)
    * 100, 1) AS q2_pct
FROM quarterly
ORDER BY q1_rev DESC NULLS LAST;
```

HW 6: Combined Pivot + Unpivot + LAG

CRAZY

SCENARIO: Build a monthly KPI report in long format showing revenue, order_count, and avg_order_value -- each with MoM growth %, ready for the BI tool.

TASK: (1) Calculate monthly KPIs in a CTE. (2) Add MoM growth using LAG. (3) Unpivot into long format: month, metric_name, current_value, mom_growth_pct.

Answer (1/2)

✅ ANSWER

```
WITH monthly AS (
  SELECT
    DATE_TRUNC('month', order_date) AS month,
    SUM(net_total)               AS revenue,
    COUNT(*)                        AS orders,
    ROUND(AVG(net_total)::NUMERIC, 2)
                                    AS aov
  FROM sales.orders
  WHERE order_status = 'Delivered'
  GROUP BY DATE_TRUNC('month', order_date)
),
with_lag AS (
  SELECT *,
    ROUND((revenue - LAG(revenue) OVER w)
      / NULLIF(LAG(revenue) OVER w, 0)
      * 100, 1) AS rev_mom,
    ROUND((orders - LAG(orders) OVER w)::NUMERIC
      / NULLIF(LAG(orders) OVER w, 0)
      * 100, 1) AS ord_mom,
    ROUND((aov - LAG(aov) OVER w)
      / NULLIF(LAG(aov) OVER w, 0)
      * 100, 1) AS aov_mom
```

Answer (2/2)

✅ ANSWER

```
  FROM monthly
  WINDOW w AS (ORDER BY month)
)
SELECT month, 'revenue' AS metric,
       revenue::NUMERIC AS value, rev_mom AS mom_pct
FROM with_lag
UNION ALL
SELECT month, 'order_count', orders, ord_mom
FROM with_lag
UNION ALL
SELECT month, 'avg_order_value', aov, aov_mom
FROM with_lag
ORDER BY month, metric;
```

KEY TAKEAWAYS

KEY TAKEAWAYS (1/2)

Pivot = rows to columns: Use SUM(CASE WHEN ...) or SUM(...) FILTER(WHERE ...) to create one column per category.

FILTER is cleaner: PostgreSQL-specific syntax that does the same as CASE WHEN but reads better.

Unpivot = columns to rows: Use UNION ALL to turn wide-format data into long-format for BI tools.

Percent-of-total pivots: Divide each FILTER column by the overall SUM to get percentages.

SQL pivot vs BI pivot:

KEY TAKEAWAYS (2/2)

Static reports: pivot in SQL. Interactive dashboards: leave long, pivot in the BI tool.

What is Next

Tomorrow we cover two powerful aggregation patterns: PERCENTILE_CONT for median/P95 calculations and DISTINCT ON for 'latest record per group' -- two patterns analysts use daily.